

Top Makinesi

Madina, IOI katılımcılarını eğlendirmek için bir top makinesi icat etmiştir. Makinenin iç yapısı şu şekildedir:

- Makine, 0'dan $N - 1$ 'e kadar numaralandırılmış N **düğüm (node)**'den oluşur. $N - 1$ düğümü **kök (root)** olarak adlandırılır.
- $0 \leq u < N - 1$ indisine sahip her u düğümü için, u düğümünün **babası (parent)**, $P[u] > u$ indisine sahip $P[u]$ düğümüdür ve u düğümü, $P[u]$ düğümünün **çocuğu (child)**'dur. Kök düğümün babası yoktur. Çocuğu olmayan bir düğüme **yaprak (leaf)** denir.

N değerini veya herhangi bir u indisindeki düğümün babasını, $P[u]$ **bilmiyorsunuz**. Bunun yerine, Madina size yaprak düğümlerin sayısının M olduğunu ve yaprak düğümlerin indislerinin $0, 1, \dots, M - 1$ olduğunu söyler. Göreviniz, makineyi çalıştırarak iç yapısını belirlemektir.

Makinenin her düğümü en fazla bir **top** tutabilir ve her topun **değeri**, negatif olmayan bir tamsayıdır. Bir düğüm, değeri x olan bir topu tutuyorsa, bu düğümün değeri x olduğunu söyleriz. Bir düğüm, içinde bir top varsa **dolu**, aksi takdirde **boş** sayılır. Başlangıçta, tüm düğümler boştur.

İki tür işlem gerçekleştirebilirsiniz:

- **insert(U, X)**: X değerine sahip yeni bir topu U yaprak düğümüne eklemeye çalışın.
 - U yaprak düğümü doluyorsa, işlem topu eklemekten **false** değerini döndürür.
 - U yaprak düğümü boşsa, top U düğümüne yerleştirilir. Ardından top, o düğümün babası boş olduğu sürece tekrar tekrar babası olan düğüme doğru hareket eder. Top, kök düğüme veya babası dolu olan bir düğüme ulaştığında hareket durur. İşlem, **true** değerini döndürür.
- **collect()**: Kök düğüm boşsa (yani makine boşsa), **collect** boş bir dizi döndürür. Aksi takdirde, aşağıda açıklanan **traverse** özyinelemeli (recursive) prosedürü, o anda makinede bulunan tüm topların değerlerini S dizisinde toplar. Başlangıçta, S dizisi boştur ve **traverse($N - 1$)** çağrılır.

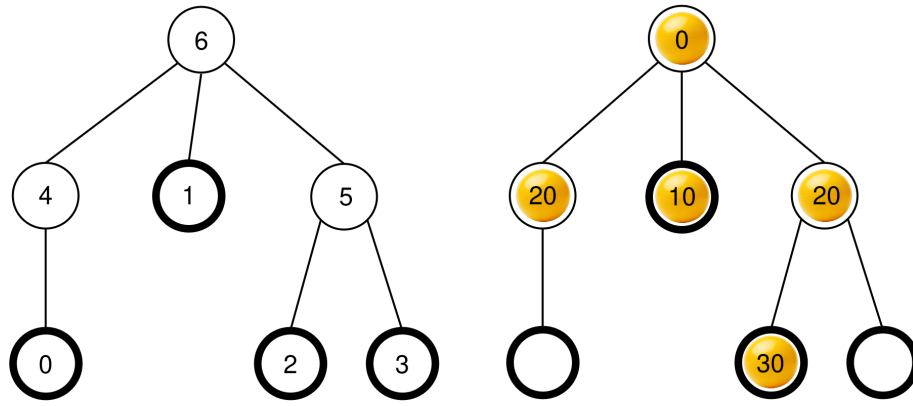
```

traverse(u):
    append the value of the ball at node u to the end of S
    let c[u] be the list of the occupied children of u
    sort c[u] in non-decreasing order according to the values
      of the balls at those nodes (if multiple nodes have the
      same value, they can appear in any order)
    for each v in c[u]:
        traverse(v)

```

Bu prosedür tamamlandıktan sonra, **tüm toplar makineden kaldırılır** ve `collect`, S değerini döndürür.

Örneğin, $N = 7$ ve $P = [4, 6, 5, 5, 6, 6]$ babalar dizisine sahip bir makineyi ele alalım. Soldaki resim, düğüm indisleriyle birlikte boş makineyi gösterirken, sağdaki resim ise bir dizi `insert` işlemi sonrasında topların olası bir düzenini göstermektedir (bu dizi Örnek bölümünde verilmiştir).



`collect()` çağrıldığında, kök düğüm (6) dolu olduğundan, S , $S = []$ olarak başlatılır ve `traverse(6)` çağrılır.

traverse(6): 6 düğümünün değeri 0'dır; bunu S 'e eklediğimizde $S = [0]$ elde edilir. 6 düğümünün çocukları 4, 1 ve 5'dir ve hepsi doludur. Bunların değerleri sırasıyla 20, 10 ve 20'dir. Azalan olmayan sırayla sıralandığında sonuç $c[6] = [1, 5, 4]$ olur (4 ve 5 düğümlerinin değeri aynı olduğu için $c[6] = [1, 4, 5]$ de geçerli olacaktır). Ardından prosedür, $c[6]$ içindeki her düğüm için bu sırayla `traverse` işlevini çağırır.

- **traverse(1):** 1 düğümünün değeri 10'dur; bunu S 'e eklediğimizde $S = [0, 10]$ elde edilir. 1 düğümünün dolu çocuğu olmadığı için bu çağrı sona erer.
- **traverse(5):** 5 düğümünün değeri 20'dir; bunu S 'e ekleyince $S = [0, 10, 20]$ elde edilir. 5 düğümünün dolu bir çocuğu vardır (2 düğümü), dolayısıyla $c[5] = [2]$ 'dir ve prosedür `traverse(2)`'yi çağırır.
 - **traverse(2):** 2 düğümünün değeri 30'dur; bunu S 'e ekleyince $S = [0, 10, 20, 30]$ elde edilir. 2 düğümünün dolu bir çocuğu yoktur, bu nedenle bu çağrı sona erer.

- Yürütme, $c[5]$ içindeki tüm çocukları işleyen `traverse(5)`'e geri döner; dolayısıyla bu çağrı sona erer.
- **`traverse(4)`**: 4 düğümünün değeri 20'dir; bunu S 'e ekleyince $S = [0, 10, 20, 30, 20]$ elde edilir. 4 düğümünün dolu çocuğu yoktur, dolayısıyla bu çağrı sona erer.

Tüm çağrılar tamamlanmıştır ve işlenecek başka düğüm kalmamıştır. Son olarak, tüm topar makineden çıkarılır ve `collect` fonksiyonu $S = [0, 10, 20, 30, 20]$ dizisini döndürür.

Göreviniz, düğüm sayısı N 'yi belirlemek ve makinenin yapısını tanımlayan, ancak yaprak olmayan ve kök olmayan düğümlerin potansiyel olarak farklı etiketlenmesine sahip bir babalar dizisi $R = [R[0], R[1], \dots, R[N-2]]$ bildirmektir. Formal olarak, bildirilen R dizisi, makinenin her bir düğüme u , $0 \leq L[u] < N$ ile farklı bir etiket $L[u]$ atanabilmesi ve aşağıdaki koşulların sağlanması durumunda doğru kabul edilir:

- Tüm $0 \leq u < M$ ve $u = N - 1$ için $L[u] = u$ olması ve
- Tüm $0 \leq u < N - 1$ için $L[P[u]] = R[L[u]]$ olması.

$R[u] > u$ olması **gerekmez**. Çözümünüzü bildirirken makine **boş olmalıdır**.

Gerçekleştirilen `collect` işlemlerinin sayısı K olsun. Tüm `insert` çağrılarında herhangi bir topun maksimum değeri B olsun. $C = K + B$ diyelim, o zaman C , 1000'i **aşmamalıdır**. Bazı alt görevlerdeki puanınız, C değerine bağlıdır.

Kodlama Detayları

Aşağıdaki prosedürü kodlamalısınız.

```
std::vector<int> find_structure(int M)
```

- M : makinedeki yaprak düğüm sayısı.
- Prosedür, her test durumu için tam olarak bir kez çağrılır.
- Prosedür, $N - 1$ uzunluğunda bir dizi $R = [R[0], R[1], \dots, R[N-2]]$ döndürmelidir; bu, doğru bir babalar dizisidir.

Makineyle etkileşim kurmak için, prosedürünüz aşağıdaki iki fonksiyonu çağırabilir.

İlk fonksiyon şudur:

```
bool insert(int U, int X)
```

- U : topun yerleştirilmesi gereken yaprak düğümün indisi. $0 \leq U < M$ koşulu karşılanmalıdır.
- X : topun tamsayı değeri. $0 \leq X \leq 1000$ koşulu karşılanmalıdır.

- Bu fonksiyon, top başarıyla yerleştirildiyse `true` değerini, U yaprak düğümü zaten doluysa `false` değerini döndürür.
- Bu fonksiyon en fazla 500 000 kez çağrılabilir.

İkinci fonksiyon şöyledir:

```
std::vector<int> collect()
```

- Eklenen tüm topların değerlerini toplar, makineyi boşaltır ve toplanan S dizisini döndürür.

Programınızın yürütülmesi sırasında herhangi bir noktada C değeri 1000 değerini aşarsa, çözümünüz `Output isn't correct: Too many resources used` dönütünü alır.

Makinenin yapısı, `find_structure` çağrılmadan önce sabitlenir. Değerlendirici, **deterministik** bir yapıya sahiptir; yani, programı iki kez çalıştırdığınızda her iki çalıştırmada da aynı işlem dizisi gerçekleştirilirse, `collect` çağrıları aynı dizileri döndürür.

Kısıtlamalar

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Alt Görevler

Alt Görev	Skor	Ek kısıtlar
1	5	Kök düğümün tam olarak M çocuğu vardır.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Hiçbir ek kısıt yoktur.

Alt görev 3 ve 4'te, skorunuz C değerine aşağıdaki şekilde bağlıdır.

Alt Görev 3 (25 puan)

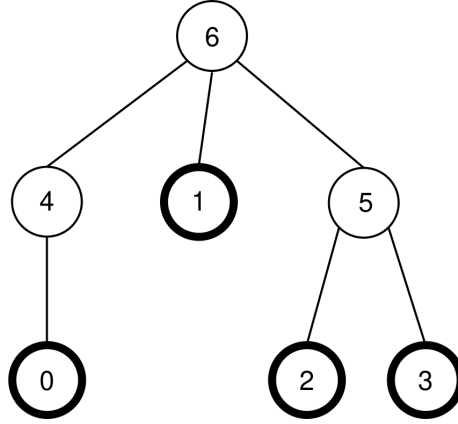
Limitler	Skor
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Alt Görev 4 (60 puan)

Limitler	Skor
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Örnek

Görev açıklamasındaki makineyi ele alalım; yani $N = 7$, $M = 4$ ve $P = [4, 6, 5, 5, 6, 6]$. Makine aşağıdaki şekilde tekrar gösterilmiştir:



Değerlendirici aşağıdaki çağrıyı yapar:

```
find_structure(4)
```

Prosedür aşağıdaki çağrı dizisini yapabilir:

1. **insert(0, 0)**: 0 yaprak düğümü boş olduğundan, 0 değerine sahip bir top 0 yaprak düğümüne yerleştirilir. $P[0] = 4$ düğümü boş olduğundan, top 4 düğümüne hareket eder. $P[4] = 6$ düğümü boş olduğundan, top 6 düğümüne hareket eder. 6 düğümü kök düğüm olduğundan, hareket durur. Çağrı, `true` değerini döndürür.
2. **insert(3, 20)**: 3 yaprak düğümü boş olduğundan, değeri 20 olan bir top 3 yaprak düğümüne yerleştirilir. $P[3] = 5$ düğümü boş olduğundan, top 5 düğümüne hareket eder. $P[5] = 6$ dolu olduğundan, hareket durur. Çağrı, `true` değerini döndürür.
3. **insert(1, 10)**: 1 yaprak düğümü boş olduğundan, değeri 10 olan bir top 1 yaprak düğümüne yerleştirilir. $P[1] = 6$ dolu olduğu için top, 1 düğümünde kalır. Çağrı, `true` sonucunu döndürür.

4. `insert(2, 30)`: 2 yaprak düğümü boş olduğundan, 30 değerindeki top 2 yaprak düğümüne yerleştirilir. $P[2] = 5$ dolu olduğu için top, 2 düğümünde kalır. Çağrı, `true` sonucunu döndürür.
5. `insert(1, 25)`: 1 yaprak düğümü dolu olduğundan top, 1 yaprak düğümüne yerleştirilmez. Çağrı, `false` sonucunu döndürür.
6. `insert(0, 20)`: 0 yaprak düğümü boş olduğundan, değeri 20 olan bir top 0 yaprak düğümüne yerleştirilir. $P[0] = 4$ düğümü boş olduğundan, top 4 düğümüne hareket eder. $P[4] = 6$ dolu olduğundan, hareket durur. Çağrı, `true` sonucunu döndürür.

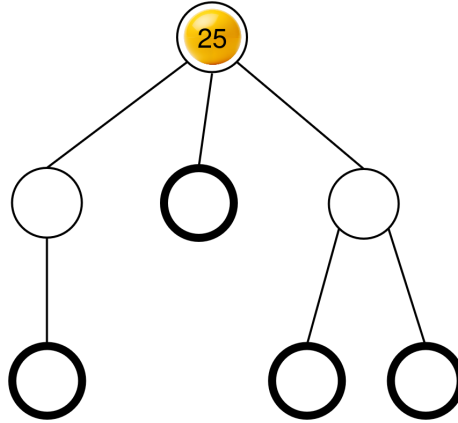
Sonuçta ortaya çıkan top konfigürasyonu, görev açıklamasında zaten gösterilmişti.

Ardından, prosedür şu çağrılarını yapabilir:

```
collect()
```

Bu çağrının yürütülmesi görev açıklamasında daha önce açıklanmıştı, dolayısıyla bu çağrı $S = [0, 10, 20, 30, 20]$ değerini döndürür. Ardından, makine tekrar boş hale gelir.

Daha sonra, prosedür `insert(2, 25)` çağrısını yapabilir. 2 yaprağı boş olduğundan, 25 değerine sahip bir top 2 yaprak düğümüne yerleştirilir. Bu top daha sonra 5 düğümüne, ardından da 6 düğümüne hareket eder ve burada durur. Çağrı, `true` değerini döndürür. Sonuçta ortaya çıkan top konfigürasyonu aşağıdaki şekilde gösterilmiştir.



Son olarak, prosedür `collect()` işlevini çağırabilir; bu işlev $S = [25]$ değerini döndürür ve makineyi boşaltır.

`find_structure`'ın dönebileceği iki doğru üst dizi vardır: $R = P = [4, 6, 5, 5, 6, 6]$ ($L = [0, 1, 2, 3, 4, 5, 6]$ etiketlemesine karşılık gelir) ve $R = [5, 6, 4, 4, 6, 6]$ ($L = [0, 1, 2, 3, 5, 4, 6]$ etiketlemesine karşılık gelir). Bu örnekte, $K = 2$ ve $B = 30$ olduğundan, $C = 32$ olur.

Örnek Değerlendirici

Girdi biçimi:

```
N M  
P[0] P[1] P[2] ... P[N-2]
```

Çıktı biçimi:

```
K B C  
Q  
R[0] R[1] R[2] ... R[Q-1]
```

Burada, Q , `find_structure` tarafından döndürülen R dizisinin uzunluğudur.